

Lab 5: Study of Addressing Modes using GDB

Aim

To study different addressing modes in C programs and analyze their behavior using GDB during program execution.

Procedure

1. Write a C program demonstrating different addressing modes such as direct, indirect (pointer), and indexed addressing.

2. Open terminal and navigate to the program directory.

3. Compile the program with debugging option:

```
gcc -g program.c -o program
```

4. Start GDB:

```
gdb ./program
```

5. Set breakpoint at main function:

```
break main
```

6. Run the program:

```
run
```

7. Execute step-by-step using:

```
next
```

8. Observe variable values:

```
print a  
print b  
print *p
```

9. Observe array access:

```
print arr[1]
```

10. Continue execution until completion:

```
continue
```

```

quit anyway? (y or n) y
(base) nitdelhi@pc:~$ nano second.c
(base) nitdelhi@pc:~$ nano secondd.c
(base) nitdelhi@pc:~$ gcc -g -O0 second.c -o second
(base) nitdelhi@pc:~$ gcc -g -O0 secondd.c -o secondd
(base) nitdelhi@pc:~$ gdb ./secondd
GNU gdb (Ubuntu 9.2-0ubuntu1-20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./secondd...
(gdb) break main
Breakpoint 1 at 0x1169: file secondd.c, line 2.
(gdb) run
Starting program: /home/nitdelhi/secondd

Breakpoint 1, main () at secondd.c:2
2      int main(){
(gdb) disassemble main
Dump of assembler code for function main:
=> 0x000055555555169 <+0>:      endbr64
0x00005555555516d <+4>:      push    %rbp
0x00005555555516e <+5>:      mov     %rsp,%rbp
0x000055555555171 <+8>:      sub     $0x40,%rsp
0x000055555555175 <+12>:     mov     %fs:0x28,%rax
0x00005555555517e <+21>:     mov     %rax,-0x8(%rbp)
0x000055555555182 <+25>:     xor     %eax,%eax
0x000055555555184 <+27>:     movl    $0x5,-0x3c(%rbp)
0x00005555555518b <+34>:     movl    $0xa,-0x38(%rbp)
0x000055555555192 <+41>:     mov     -0x3c(%rbp),%edx
0x000055555555195 <+44>:     mov     -0x38(%rbp),%eax
0x000055555555198 <+47>:     add     %edx,%eax
0x00005555555519a <+49>:     mov     %eax,-0x34(%rbp)
0x00005555555519d <+52>:     movl    $0xa,-0x20(%rbp)
0x0000555555551a4 <+59>:     movl    $0x14,-0x1c(%rbp)
0x0000555555551ab <+66>:     movl    $0x1e,-0x18(%rbp)
0x0000555555551b2 <+73>:     movl    $0x28,-0x14(%rbp)
0x0000555555551b9 <+80>:     lea     -0x20(%rbp),%rax
0x0000555555551bd <+84>:     mov     %rax,-0x28(%rbp)
0x0000555555551c1 <+88>:     mov     -0x28(%rbp),%rax
0x0000555555551c5 <+92>:     mov     0x8(%rax),%eax
0x0000555555551c8 <+95>:     mov     %eax,-0x30(%rbp)
0x0000555555551cb <+98>:     mov     -0x14(%rbp),%eax
0x0000555555551ce <+101>:    mov     %eax,-0x2c(%rbp)
0x0000555555551d1 <+104>:    mov     -0x2c(%rbp),%ecx

0x000055555555198 <+47>:     add     %edx,%eax
0x00005555555519a <+49>:     mov     %eax,-0x34(%rbp)
0x00005555555519d <+52>:     movl    $0xa,-0x20(%rbp)
0x0000555555551a4 <+59>:     movl    $0x14,-0x1c(%rbp)
0x0000555555551ab <+66>:     movl    $0x1e,-0x18(%rbp)
0x0000555555551b2 <+73>:     movl    $0x28,-0x14(%rbp)
0x0000555555551b9 <+80>:     lea     -0x20(%rbp),%rax
0x0000555555551bd <+84>:     mov     %rax,-0x28(%rbp)
0x0000555555551c1 <+88>:     mov     -0x28(%rbp),%rax
0x0000555555551c5 <+92>:     mov     0x8(%rax),%eax
0x0000555555551c8 <+95>:     mov     %eax,-0x30(%rbp)
0x0000555555551cb <+98>:     mov     -0x14(%rbp),%eax
0x0000555555551ce <+101>:    mov     %eax,-0x2c(%rbp)
0x0000555555551d1 <+104>:    mov     -0x2c(%rbp),%ecx
0x0000555555551d4 <+107>:    mov     -0x30(%rbp),%edx
0x0000555555551d7 <+110>:    mov     -0x34(%rbp),%eax
0x0000555555551da <+113>:    mov     %eax,%rsi
0x0000555555551dc <+115>:    lea     0xe21(%rip),%rdi      # 0x55555555004
0x0000555555551e3 <+122>:    mov     $0x0,%eax
0x0000555555551e8 <+127>:    callq   0x55555555070 <printf@plt>
0x0000555555551ed <+132>:    mov     $0x0,%eax
0x0000555555551f2 <+137>:    mov     -0x8(%rbp),%rsi
0x0000555555551f6 <+141>:    xor     %fs:0x28,%rsi
0x0000555555551ff <+150>:    je      0x55555555206 <main+157>
0x000055555555201 <+152>:    callq   0x55555555060 <__stack_chk_fail@plt>
0x000055555555206 <+157>:    leaveq   %rsp
0x000055555555207 <+158>:    retq

End of assembler dump.
(gdb) info registers
rax      0x55555555169      93824992235881
rbx      0x55555555210      93824992236048
rcx      0x55555555210      93824992236048
rdx      0x7fffffffdeb8     140737488346808
rsi      0x7fffffffdea8     140737488346792
rdi      0x1                1
rbp      0x0                0x0
rsp      0x7fffffffddb8     0x7fffffffddb8
r8        0x0                0
r9        0x7ffff7fe0d60     140737354009952
r10       0x13              19
r11       0x2                2
r12       0x55555555080      93824992235648
r13       0x7fffffffdea0     140737488346784
r14       0x0                0
r15       0x0                0
rip       0x55555555169      0x55555555169 <main>
eflags    0x246             [ PF ZF IF ]
cs        0x33              51
ss        0x2b              43
ds        0x0                0
es        0x0                0
fs        0x0                0
gs        0x0                0
(gdb) quit
A debugging session is active.

```

Result

The different addressing modes were successfully studied using GDB.

- Direct addressing was observed through normal variable access.
- Indirect addressing was demonstrated using pointers.
- Indexed addressing was verified using array elements.

The experiment helped in understanding how different addressing modes access data during program execution.